

**Proiect Microcontrolere:
Sistem Embedded de Securitate cu
Autentificare Multi-Factor (2FA) și
Răspuns Activ**

Roibu Vlad-Constantin, anul II, seria A, semigrupa 1.1

I.Descrierea Funcționării Sistemului

Sistemul reprezintă un prototip funcțional al unei console de securitate pentru seifuri de înaltă siguranță, bazat pe microcontrolerul ATmega328P. Arhitectura software este construită în jurul unei mașini cu stări finite (FSM - Finite State Machine), ceea ce garantează o rulare fluidă, non-blocantă și un control predictibil al perifericelor.

Logica de control a sistemului a fost implementată utilizând modelul matematic al unei Mașini cu Stări Finite (Finite State Machine). Această decizie arhitecturală a fost luată pentru a asigura un comportament determinist, stabil și non-blocant al microcontrolerului pe durata rulării.

Spre deosebire de o execuție liniară clasică, în care resursele procesorului sunt adesea irosite prin bucle de așteptare, modelul FSM împarte funcționarea sistemului în stări operaționale mutual exclusive (de exemplu: **BLOCAT**, **ASTEPTARE_MAGNET**, **DEBLOCAT**, **MOD_APARARE**). La orice moment de execuție, sistemul se află într-o singură stare activă.

Tranzițiile între aceste stări sunt condiționate și declanșate strict de evenimente specifice: externe (introducerea unei secvențe corecte de caractere pe interfața matricială, declanșarea întreruperii hardware de către senzorul Hall) sau interne (atingerea pragului de încercări eșuate, care forțează trecerea în modul defensiv). Acest design garantează că arhitectura embedded este permanent receptivă la senzorii critici și operează cu un timp de răspuns optim.

Proiectul implementează un mecanism de autentificare în doi pași (Two-Factor Authentication - 2FA), persistența datelor în memorie nevolatilă și un sistem activ de apărare împotriva intruziunilor. Funcționarea sistemului este divizată în următoarele etape logice:

1. Starea de Repaus și Izolare (Modul BLOCAT)

La alimentarea sistemului, microcontrolerul inițializează perifericele și citește parola master din memoria EEPROM internă. Dacă memoria este neinițializată, se scrie automat un PIN implicit (1234). În această stare, actuatorul mecanic (servomotorul) este forțat în poziția de repaus (zăvor închis), LED-ul RGB emite o lumină roșie continuă pentru a indica restricția accesului, iar pe ecranul LCD I2C este afișat mesajul de întâmpinare. Sistemul scanează continuu matricea tastaturii 4x4.

2. Autentificarea Primară (Validarea PIN-ului)

Utilizatorul introduce un cod format din 4 cifre. Pentru securitate vizuală, caracterele introduse sunt mascate pe ecran sub formă de asteriscuri (*), iar fiecare apăsare este acompaniată de un scurt feedback acustic generat de buzzer. Prin apăsarea tastei de confirmare (#), sistemul compară buffer-ul introdus cu valoarea stocată în EEPROM:

- **În caz de succes:** Sistemul avansează în starea de așteptare pentru al doilea factor de securitate.
- **În caz de eroare:** Sistemul contorizează încercarea eșuată și emite un avertisment sonor lung.

3. Autentificarea Secundară (Întreruperi Hardware)

Odată introdus PIN-ul corect, interfața vizuală își schimbă starea (LED albastru) și solicită cheia fizică. Această etapă utilizează un senzor magnetic cu efect Hall. Pentru a asigura un răspuns în timp real și a demonstra stăpânirea arhitecturii low-level a microcontrolerului, senzorul nu este interogată prin funcții clasice de tip *polling*, ci declanșează o întrerupere hardware externă (INT0). Configurația este realizată prin manipularea directă a regiștrilor **EICRA** și **EIMSK**, declanșarea având loc pe front descrescător (*falling edge*). Odată detectat câmpul magnetic, rutina de tratare a întreruperii (ISR) modifică un *flag* de stare volatil, permițând mașinii de stări să deblocheze seiful.

4. Acționarea Mecanică (Modul DEBLOCAT)

Validarea ambilor factori de securitate declanșează deschiderea seifului. LED-ul RGB comută pe culoarea verde, iar servomotorul cu rotație continuă (360°) primește un impuls calculat temporal pentru a retrage zăvorul mecanic. Sistemul rămâne în această stare timp de 5 secunde, după care zăvorul este împins automat înapoi în poziția de blocare prin inversarea sensului de rotație a servomotorului, asigurând reînarmarea sistemului fără intervenția utilizatorului.

5. Meniul de Administrare (Schimbarea Parolei)

Sistemul este complet autonom și nu necesită reconectarea la un mediu de dezvoltare pentru administrare. Prin apăsarea tastei **A** din starea de repaus, utilizatorul poate accesa meniul de schimbare a PIN-ului. Pentru a preveni modificările neautorizate, introducerea unei noi parole este permisă doar după reverificarea parolei curente. Noul cod este salvat utilizând funcția de *update* a memoriei EEPROM, metodă care scrie datele doar dacă acestea diferă de cele existente, optimizând astfel durata de viață a celulelor de memorie.

6. Modul de Apărare Activă (Anti-Intruziune)

Sistemul include un mecanism de tip *failsafe* pentru a descuraja atacurile de tip *brute-force*. Variabila care contorizează introducerile eronate de PIN declanșează o stare de alarmă extremă la a treia greșeală consecutivă. În starea **MOD_APARARE**, sistemul blochează complet citirea tastaturii, aprinde un actuator vizual de înaltă intensitate (Modul Laser) îndreptat către zona de acțiune și generează o sirenă de mare putere alternând frecvențele pe buzzer. Această stare persistă pentru un ciclu predefinit, după care sistemul se resetează și revine la starea de blocare.

II. Configurația Pinilor

1. Interfața I2C și Senzorul Principal (Pini Speciali)

- **Senzor Magnetic (Hall):**
 - Semnal (OUT / DO) → **Pin Digital 2** (*Critic: Acesta este pinul pentru întreruperea hardware INT0*)
 - VCC → 5V
 - GND → GND
- **Modul LCD I2C:**
 - SDA (Date) → **Pin Analogic A4**
 - SCL (Ceas) → **Pin Analogic A5**
 - VCC → 5V
 - GND → GND

2. Acționare și Semnalizare (leșiri)

- **Modul Laser (KY-008):**
 - Semnal (S / VCC) → **Pin Digital 13**
 - GND (-) → GND
- **Servomotor SG90 (360°):**
 - Semnal (Fir Galben/Portocaliu) → **Pin Digital 11**
 - VCC (Fir Roșu) → 5V
 - GND (Fir Maro/Negru) → GND
- **Buzzer Activ:**
 - Plus (+) → **Pin Digital 12**
 - Minus (-) → GND
- **Modul LED RGB (Catod Comun):**
 - R (Roșu) → **Pin Analogic A0**
 - G (Verde) → **Pin Analogic A1**
 - B (Albastru) → **Pin Analogic A2**
 - GND (-) → GND

3. Tastatura Matricială 4x4 (Intrări)

Ordinea pinilor de pe panglică (de la stânga la dreapta):

- **Rânduri (Rows):**
 - R1 → **Pin Digital 10**
 - R2 → **Pin Digital 9**
 - R3 → **Pin Digital 8**
 - R4 → **Pin Digital 7**
- **Coloane (Columns):**
 - C1 → **Pin Digital 6**
 - C2 → **Pin Digital 5**
 - C3 → **Pin Digital 4**
 - C4 → **Pin Digital 3**

III. Componente și descrierea acestora

Link-uri de unde am procurat componentele:

<https://ty.gl/f9kgc8jut7tr7>

<https://ty.gl/zned98d0ywwbi>

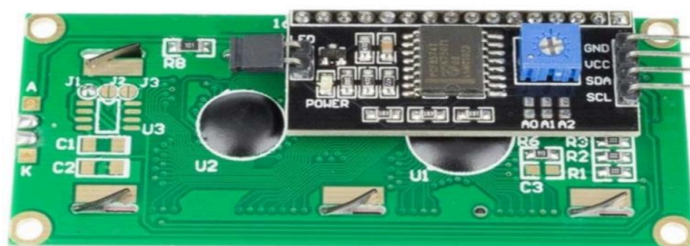
<https://ty.gl/35yjrzh3kw1j7>

1. Unitatea Centrală de Procesare: Placa de Dezvoltare (Clonă Arduino Uno CH340)

- **Microcontroler:** Baza sistemului este microcontrolerul **ATmega328P** pe arhitectură AVR (8 biți, tip RISC). Acesta rulează la o frecvență de 16 MHz, oferind suficientă putere de calcul pentru menținerea stabilă a mașinii de stări (FSM) fără întârzieri perceptibile.
- **Memorie Nevolatilă (EEPROM):** Placa dispune de 1 KB de memorie EEPROM internă. În acest proiect, memoria este utilizată pentru persistența datelor critice (salvarea parolei master), garantând reținerea codului PIN chiar și în cazul întreruperii totale a alimentării externe.
- **Interfața USB-Serial:** Utilizarea variantei SMD echipată cu chip-ul **CH340** pentru conversia USB-to-Serial asigură o comunicare stabilă pentru încărcarea firmware-ului și permite costuri reduse de implementare, fiind o soluție preferată în prototiparea sistemelor integrate.

2. Modulul de Afișaj: LCD 1602 cu Interfață I2C (PCF8574)

- **Afișajul Liquid Crystal (LCD):** Dispune de o matrice de 16 coloane și 2 rânduri, având controlerul standard integrat (tipic Hitachi HD44780).
- **Expansorul de I/O (I2C):** Elementul inovator al acestui sub-ansamblu este modulul adaptor (bazat pe circuitul integrat PCF8574). Acesta convertește datele seriale primite pe magistrala **I2C** (liniile SDA și SCL) în date paralele necesare LCD-ului. Această abordare reduce numărul pinilor necesari pentru comunicare de la minim 6 pini digitali la doar 2 pini analogici, eliberând resurse hardware vitale pentru restul sistemului.



3. Matricea de Intrare: Tastatură Matricială 4x4

- **Principiu de Funcționare:** Tastatura este o matrice pasivă formată din 16 butoane (push-buttons) aranjate în 4 rânduri și 4 coloane, fără circuite integrate active proprii.
- **Metoda de Interogare (Scanning):** Microcontrolerul scanează tastatura aplicând un semnal logic **LOW** secvențial pe fiecare coloană și citind starea rândurilor. Pinii rândurilor sunt menținuți la o stare logică **HIGH** internă prin activarea rezistențelor de *pull-up* din microcontroler. La apăsarea unui buton, se creează un scurtcircuit între un rând și o coloană, permițând software-ului să decodifice coordonata exactă a tastei apăsate.



4. Senzorul Magnetic de Validare (Factorul 2): Modul cu Efect Hall

- **Principiul Efectului Hall:** Modulul folosește un traductor care, în prezența unui câmp magnetic extern perpendicular, generează o diferență de potențial proporțională.
- **Condiționarea Semnalului:** Modulul include un comparator integrat (de obicei un LM393) și un potențiometru pentru ajustarea sensibilității, transformând semnalul analogic brut într-un semnal logic digital clar (**HIGH/LOW**).
- **Integrarea în Sistem:** Ieșirea digitală a senzorului este conectată pe o linie de întrerupere hardware (INT0), evitând interogarea constantă (polling) și garantând o reacție instantanee din partea procesorului.



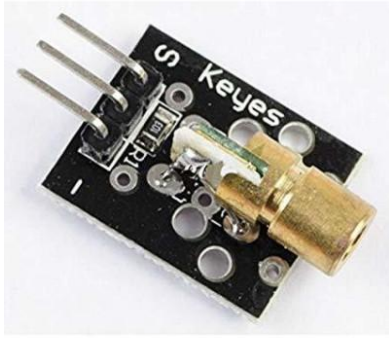
5. Actuatorul Mecanic: Servomotor SG90

- **Diferența Față de Servomotoarele Standard:** Spre deosebire de un servomotor clasic care folosește potențiometre interne pentru a menține un unghi fix (0° - 180°), varianta 360° funcționează ca un motor DC cu reductor, unde semnalul de control dictează *viteza* și *sensul de rotație*.
- **Controlul prin Semnal PWM:** Microcontrolerul generează un semnal Pulse Width Modulation (PWM). În logica de control a bibliotecii `<Servo.h>`, scrierea valorii de `90` trimite un puls de ~ 1.5 ms (semnal de Stop/Punct mort). Valori inferioare trag zăvorul într-o direcție (deschidere), iar valori superioare îl împing în direcția opusă (închidere), controlul fiind bazat pe durata menținerii semnalului.



6. Modulul Defensiv Optic: Modul Diodă Laser

- **Specificații:** Modulul este echipat cu o diodă laser de culoare roșie (lungime de undă tipică de 650 nm) cu o putere de ieșire de 5 mW.
- **Control Electronic:** Este o componentă simplă, cu o rezistență de limitare integrată direct pe placa (PCB-ul) modulului, ceea ce permite conectarea sigură, în regim de curent direct, la pinul digital de ieșire al microcontrolerului, fără a risca deteriorarea portului I/O prin depășirea curentului maxim suportat (tipic 20 mA - 40 mA pe ATmega328P).



7. Actuatorele de Semnalizare: Modul LED RGB și Buzzer Activ

- **Modul LED RGB (Catod Comun):** Este un dispozitiv optoelectronic ce încorporează trei diode emițătoare de lumină distincte (Roșu, Verde, Albastru) într-o singură capsulă, având pinul negativ (GND) comun. Aplicarea de tensiuni pe pinii R, G, B generează culorile de stare.



- **Buzzer Activ:** Componenta conține propriul oscilator electronic intern. La aplicarea unei tensiuni continue (5V), acesta generează automat un semnal acustic la o frecvență fixă (tipic 2 kHz). Utilizarea funcției `tone()` peste acesta modulează suplimentar alimentarea, creând efectul de sirenă alternantă utilizat în starea de alarmă.



IV.Codul (Alternativ,link către

GitHub:<https://github.com/vladroibu2005/Proiect-Seif-Arduino>)

```
#include <Wire.h>                // Biblioteca pentru comunicarea I2C
(folosită de display)
#include <LiquidCrystal_I2C.h>    // Biblioteca pentru controlul
display-ului LCD prin I2C
#include <Keypad.h>               // Biblioteca pentru citirea tastaturii
matriciale
#include <Servo.h>                // Biblioteca pentru controlul
servomotorului
#include <avr/interrupt.h>        // Biblioteca specifică AVR pentru
manipularea întreruperilor hardware
#include <EEPROM.h>               // Biblioteca pentru stocarea
permanentă a PIN-ului în memorie

// --- Configurari Hardware ---
LiquidCrystal_I2C lcd(0x27, 16, 2); // Inițializare LCD cu adresa I2C
0x27, 16 coloane și 2 rânduri
Servo usaSeif;                  // Crearea obiectului pentru
controlul servomotorului

const byte pinSenzor = 2; // Pin digital 2 pentru INT0 (întrerupere
hardware pentru Senzor Magnetic)
const byte pinServo = 11; // Pin digital 11 cu suport PWM pentru
semnalul servomotorului
const byte pinBuzzer = 12; // Pin digital 12 pentru controlul
difuzorului (buzzer)
const byte pinLaser = 13; // Pinul 13 pentru controlul modulului Laser
(folosit la alarmă)

// Pini LED RGB (Catod Comun)
const byte pinR = A0; // Pin analogic folosit ca digital pentru
culoarea Roșu
const byte pinG = A1; // Pin analogic folosit ca digital pentru
culoarea Verde
const byte pinB = A2; // Pin analogic folosit ca digital pentru
culoarea Albastru

// --- Configurarea Tastaturii ---
const byte RANDURI = 4; // Tastatura are 4 rânduri
const byte COLOANE = 4; // Tastatura are 4 coloane
char taste[RANDURI][COLOANE] = {
```

```

    {'1','2','3','A'},    // Maparea caracterelor de pe tastatură -
Rândul 1
    {'4','5','6','B'},    // Maparea caracterelor de pe tastatură -
Rândul 2
    {'7','8','9','C'},    // Maparea caracterelor de pe tastatură -
Rândul 3
    {'*','0','#','D'}     // Maparea caracterelor de pe tastatură -
Rândul 4
};
byte piniRanduri[RANDURI] = {10, 9, 8, 7}; // Pinii conectați la
rândurile R1, R2, R3, R4
byte piniColoane[COLOANE] = {6, 5, 4, 3}; // Pinii conectați la
coloanele C1, C2, C3, C4
// Inițializarea obiectului tastatură pe baza mapării și a pinilor
declarați
Keypad tastatura = Keypad(makeKeymap(taste), piniRanduri, piniColoane,
RANDURI, COLOANE);

// --- Masina de Stari (FSM - Finite State Machine) ---
enum StareSistem {
    BLOCAT,                // Starea implicită, seiful este închis și
așteaptă PIN-ul
    ASTEPTARE_MAGNET,      // PIN-ul a fost corect, se așteaptă validarea
cu senzorul magnetic (2FA)
    DEBLOCAT,              // Acces permis, servomotorul deschide seiful
    SCHIMBARE_PIN_VECHI,   // Starea de introducere a PIN-ului curent
pentru verificare înainte de modificare
    SCHIMBARE_PIN_NOU,     // Starea în care se introduce noul PIN
    MOD_APARARE             // Starea de alarmă declanșată de prea multe
încercări greșite
};
StareSistem stareCurenta = BLOCAT; // Setăm starea inițială la pornirea
sistemului

// --- Variabile ---
String pinSalvat = "";      // Variabilă pentru a stoca PIN-ul citit din
memoria EEPROM
String pinIntrodus = "";    // Variabilă în care se construiește PIN-ul
tastat de utilizator
volatile bool magnetDetectat = false; // Variabilă modificată în
interiorul ISR; trebuie declarată 'volatile'
int incercariGresite = 0;    // Contor pentru a bloca sistemul după 3
încercări eșuate

```

```

// --- RUTINA DE INTRERUPERE (ISR - Interrupt Service Routine) ---
// Funcția care se execută automat când se declanșează întreruperea
INT0 (pin 2)
ISR(INT0_vect) {
    // Magnetul validează deschiderea DOAR dacă sistemul a trecut deja de
    pasul cu PIN-ul
    if (stareCurenta == ASTEPTARE_MAGNET) {
        magnetDetectat = true; // Setăm semnalizatorul (flag) că senzorul a
        citit magnetul
    }
}

void setup() {
    // --- Initializare EEPROM ---
    // Dacă EEPROM-ul este gol (255 e valoarea default la un chip nou),
    scriem un PIN inițial "1234"
    if (EEPROM.read(0) == 255) {
        EEPROM.update(0, '1');
        EEPROM.update(1, '2');
        EEPROM.update(2, '3');
        EEPROM.update(3, '4');
    }

    // Citim cele 4 caractere din EEPROM și le concatenăm în variabila
    pinSalvat
    for (int i = 0; i < 4; i++) {
        pinSalvat += (char)EEPROM.read(i);
    }

    // --- Initializare Periferice ---
    lcd.init();           // Inițializarea ecranului LCD
    lcd.backlight();      // Aprinderea luminii de fundal a ecranului

    usaSeif.attach(pinServo); // Conectăm servomotorul la pinul 11
    usaSeif.write(90);       // Setăm servo-ul la 90 de grade (punctul
    de STOP pentru servo-urile cu rotație continuă 360)

    pinMode(pinBuzzer, OUTPUT); // Configurăm pinul buzzer-ului ca ieșire
    pinMode(pinLaser, OUTPUT);  // Configurăm pinul laserului ca ieșire
    digitalWrite(pinLaser, LOW); // Ne asigurăm că laserul este stins la
    pornire

```

```

pinMode(pinR, OUTPUT); // Configurare pin componentă Roșie LED
pinMode(pinG, OUTPUT); // Configurare pin componentă Verde LED
pinMode(pinB, OUTPUT); // Configurare pin componentă Albastră LED

pinMode(pinSenzor, INPUT_PULLUP); // Senzorul pe pin 2 cu rezistență
internă de pull-up activată

// --- CONFIGURARE REGISTRE PENTRU INT0 (Hardware Interrupts) ---
cli(); // Dezactivăm global întreruperile cât timp configurăm
registrii
EICRA |= (1 << ISC01); // Setăm bitul ISC01 din registrul de
control (EICRA)
EICRA &= ~(1 << ISC00); // Ștergem bitul ISC00 -> Configurația
înseamnă declanșare pe "Falling edge" (trecere din HIGH în LOW)
EIMSK |= (1 << INT0); // Activăm masca de întrerupere pentru INT0
(pinul 2)
sei(); // Reactivăm global întreruperile
// -----

afiseazaEcranBlocat(); // Afișăm ecranul principal de așteptare
}

void loop() {
    char tasta = tastatura.getKey(); // Citim apăsarea curentă de pe
tastatură

    // Verificăm în ce stare se află sistemul pentru a ști cum
interpretăm apăsările
    switch (stareCurenta) {

        case BLOCAT: // Așteptăm introducerea PIN-ului
            if (tasta) {
                if (tasta == '*') { // Tasta '*' funcționează ca un buton de
ștergere (Resetare input)
                    pinIntrodus = "";
                    afiseazaEcranBlocat();
                }
                else if (tasta == '#') { // Tasta '#' este folosită ca "Enter"
pentru a confirma PIN-ul introdus
                    verificaPIN();
                }
                else if (tasta == 'A') { // Tasta 'A' declanșează procesul de
schimbare a PIN-ului

```

```

        stareCurenta = SCHIMBARE_PIN_VECHI; // Trecem în starea de
schimbare a PIN-ului
        pinIntrodus = ""; // Curățăm buffer-ul
        seteazaRGB(LOW, LOW, HIGH); // Setăm LED-ul pe
Albastru
        lcd.clear();
        lcd.print("PIN-ul Vechi:");
        lcd.setCursor(0, 1); // Mutăm cursorul pe a
doua linie a LCD-ului
    }
    else {
        // Prevenim overflow-ul dacă cineva apasă prea multe taste
(limită 16 caractere)
        if (pinIntrodus.length() < 16) {
            pinIntrodus += tasta; // Adăugăm tasta la șirul PIN-ului
            lcd.print('*'); // Afișăm o stelută pentru a masca
PIN-ul real
            emiteSUNET(50); // Bip scurt la fiecare apăsare de
tastă
        }
    }
}
break;

case SCHIMBARE_PIN_VECHI: // Solicităm PIN-ul vechi pentru a aproba
modificarea
    if (tasta) {
        if (tasta == '*') { // Anulare acțiune
            stareCurenta = BLOCAT;
            pinIntrodus = "";
            afiseazaEcranBlocat();
        }
        else if (tasta == '#') { // Validăm dacă PIN-ul vechi e corect
            if (pinIntrodus == pinSalvat) {
                stareCurenta = SCHIMBARE_PIN_NOU; // Trecem la pasul
următor: setarea noului PIN
                pinIntrodus = "";
                lcd.clear();
                lcd.print("PIN Nou (4 cif):");
                lcd.setCursor(0, 1);
                emiteSUNET(200); // Bip lung de confirmare a parolei vechi
            } else {
                // Dacă s-a greșit PIN-ul vechi, abandonăm procesul

```

```

        lcd.clear();
        lcd.print("PIN Incorect!");
        emiteSunet(500); delay(2000); // Bip de eroare și pauză de
2 secunde

        stareCurenta = BLOCAT;
        pinIntrodus = "";
        afiseazaEcranBlocat();
    }
}
else {
    // Tastare normală de cifre, mascate cu steluțe
    if (pinIntrodus.length() < 16) {
        pinIntrodus += tasta;
        lcd.print('*');
        emiteSunet(50);
    }
}
}
break;

case SCHIMBARE_PIN_NOU: // Salvăm un PIN nou în EEPROM
    if (tasta) {
        if (tasta == '*') { // Anulare acțiune
            stareCurenta = BLOCAT;
            pinIntrodus = "";
            afiseazaEcranBlocat();
        }
        else if (tasta == '#') { // Confirmare noul PIN
            if (pinIntrodus.length() == 4) { // Ne asigurăm că PIN-ul are
exact 4 caractere
                // Scriem noile 4 caractere peste primele 4 adrese din
memoria EEPROM
                for (int i = 0; i < 4; i++) {
                    EEPROM.update(i, pinIntrodus[i]); // .update() scrie doar
dacă valoarea e diferită (salvează cicluri de scriere)
                }
                pinSalvat = pinIntrodus; // Actualizăm și variabila din
memoria RAM

                lcd.clear();
                lcd.print("PIN Salvat!");
                emiteSunet(100); delay(50); emiteSunet(100); delay(1500);
// Sunet de succes (dublu bip)

```

```

        stareCurenta = BLOCAT; // Revenim la ecranul de start
        pinIntrodus = "";
        afiseazaEcranBlocat();
    } else {
        // Dacă PIN-ul nou are mai mult sau mai puțin de 4 cifre
        lcd.clear();
        lcd.print("Eroare! Doar 4.");
        emiteSUNET(500); delay(2000);
        lcd.clear();
        lcd.print("PIN Nou (4 cif):");
        lcd.setCursor(0, 1);
        pinIntrodus = ""; // Golim buffer-ul pentru a încerca din
nou
    }
}
else {
    // Aici PIN-ul nou este vizibil în timp ce se tastează, cu
limită de 4 caractere
    if (pinIntrodus.length() < 4) {
        pinIntrodus += tasta;
        lcd.print(tasta); // Afișăm tasta efectiv pe ecran
        emiteSUNET(50);
    }
}
break;

case ASTEPTARE_MAGNET: // PIN introdus corect, așteptăm factorul 2
(Senzorul Hall)
    if (magnetDetectat) { // Variabila devine 'true' în funcția ISR
când magnetul este apropiat de senzor
        magnetDetectat = false; // Resetăm flag-ul pentru utilizări
viitoare
        stareCurenta = DEBLOCAT; // Trecem în starea finală de succes
        deschideSeif(); // Apelăm funcția care învâрте
servomotorul
    }
    if (tasta == '*') { // Dacă utilizatorul se răzgândește sau
renunță, tasta '*' anulează pasul 2
        stareCurenta = BLOCAT;
        pinIntrodus = "";
        afiseazaEcranBlocat();
    }
}

```

```

        break;

    case DEBLOCAT: // Starea în care ușa este menținută deschisă
        delay(5000); // Seiful stă deschis timp de 5 secunde

        usaSeif.write(0); // Servomotorul se mișcă în direcție inversă
        pentru a încuia
        delay(500); // Se lasă servo-ul să se rotească 500ms
        usaSeif.write(90); // Oprește servomotor

        emiteSunet(100); emiteSunet(100); // Sunet de
        finalizare/închidere

        stareCurenta = BLOCAT; // Resetăm la starea inițială
        pinIntrodus = "";
        afiseazaEcranBlocat();
        break;

    case MOD_APARARE: // S-au introdus 3 PIN-uri greșite
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("ALARM INTRUS!");
        lcd.setCursor(0, 1);
        lcd.print("SISTEM BLOCAT");

        digitalWrite(pinLaser, HIGH); // Aprindem modulul Laser

        // Buclă pentru un efect de Sirena vizuală și auditivă de aprox 4
        secunde
        for (int i = 0; i < 10; i++) {
            seteazaRGB(HIGH, LOW, LOW); // LED-ul luminează Roșu aprins
            tone(pinBuzzer, 1000); // Sunet frecvență înaltă
            delay(200);
            seteazaRGB(LOW, LOW, LOW); // Stingem LED-ul
            tone(pinBuzzer, 500); // Sunet frecvență joasă (efect de
            tip "weeo-weeo")
            delay(200);
        }

        noTone(pinBuzzer); // Oprim sunetul complet
        digitalWrite(pinLaser, LOW); // Stingem laserul

```



```

        incercariGresite = 0; // Resetăm contorul de erori pentru a oferi
din nou o șansă
        stareCurenta = BLOCAT; // Întoarcere la așteptarea PIN-ului
        pinIntrodus = "";
        afiseazaEcranBlocat();
        break;
    }
}

// --- Functii Auxiliare ---

// Funcție pentru controlul simplificat al LED-ului RGB
void seteazaRGB(bool r, bool g, bool b) {
    digitalWrite(pinR, r); // Aprinde/stinge componenta Roșie
    digitalWrite(pinG, g); // Aprinde/stinge componenta Verde
    digitalWrite(pinB, b); // Aprinde/stinge componenta Albastră
}

// Funcție pentru afișarea interfeței standard de bază (LED Roșu și
mesaj LCD)
void afiseazaEcranBlocat() {
    seteazaRGB(HIGH, LOW, LOW); // Starea de repaus, semnalizată prin LED
Roșu
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Introduceti PIN:");
    lcd.setCursor(0, 1);          // Poziționează cursorul pe linia a 2-a
    pentru introducerea caracterelor
}

// Funcție care verifică parola și contorizează greșelile
void verificaPIN() {
    lcd.clear();
    lcd.setCursor(0, 0);

    if (pinIntrodus == pinSalvat) { // Verificare condiție de succes
        incercariGresite = 0;          // Resetăm contorul la un PIN corect
        stareCurenta = ASTEPTARE_MAGNET; // Avansăm mașina de stări la
Pasul 2
        magnetDetectat = false;        // Curățăm preventiv flag-ul
senzorului
        seteazaRGB(LOW, LOW, HIGH);    // LED Albastru pentru stadiul
intermediar (2FA)
    }
}

```

```

    lcd.print("PIN OK. Pas 2:");
    lcd.setCursor(0, 1);
    lcd.print("Scanati Magnet!");
    emiteSunet(200);
} else {
    incercariGresite++; // Incrementăm greșelile

    // Verificăm dacă s-a atins limita de siguranță
    if (incercariGresite >= 3) {
        stareCurenta = MOD_APARARE; // Sistemul intră în alarma
antiefrație
        pinIntrodus = "";
    } else {
        // Afișează numărul de încercări rămase înainte de alarmă
        lcd.print("PIN Incorect!");
        lcd.setCursor(0, 1);
        lcd.print("Incerari: ");
        lcd.print(3 - incercariGresite);
        emiteSunet(500);
        delay(2000); // Ține ecranul cu eroare vizibil 2 secunde
        pinIntrodus = "";
        afiseazaEcranBlocat();
    }
}
}

// Funcția de declanșare a motorului pentru deschiderea efectivă a
seifului
void deschideSeif() {
    seteazaRGB(LOW, HIGH, LOW); // LED Verde - Succes total
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("ACCES APROBAT");
    lcd.setCursor(0, 1);
    lcd.print("Seiful e deschis");

    // Arpegiu scurt de aprobare a accesului
    emiteSunet(100); delay(50); emiteSunet(150); delay(50);
    emiteSunet(300);

    usaSeif.write(180); // Învâрте servomotorul pentru a debloca yala (pe
directia 'înainte')
    delay(500);          // Se învâрте 500 de milisecunde

```

```

    usaSeif.write(90); // Oprește servomotorul (la cele cu rotație
    continuă, 90 înseamnă repaus)
}

// Funcție utilitară pentru a acționa buzzer-ul pentru o anumită durată
void emiteSUNET(int durata) {
    digitalWrite(pinBuzzer, HIGH); // Pornește difuzorul
    delay(durata);                  // Așteaptă X milisecunde
    digitalWrite(pinBuzzer, LOW);  // Oprește difuzorul
}

```

V.Poză proiect propriu-zis

